

## Konvencie pseudokódov

Ukážky kódov zámerne nedodržiavajú syntax žiadneho programovacieho jazyka. Zvolili sme jazykovo nezávislú formu zápisu, využívajúcu slovenské výrazy pre kľúčové slová. Veríme, že táto forma zápisu umožní na jednej strane jasne pochopiť uvedené definície premenných, rozhraní a tried, ako aj činnosť popisovaných algoritmov a na druhej strane nebude zväzovať konvenciami či obmedzeniami konkrétného programovacieho jazyka. Pseudokód nepopisuje všetky potrebné implementačné detaily (tie je potrebné doplniť pri implementácii v konkrétnom programovacom jazyku), napriek tomu je prevod pseudokódov do zvoleného programovacieho jazyka jednoduchý. Pre čo najväčšiu názornosť a prehľadnosť sme zvolili nasledujúcu konvenciu.

Primitívne typy označujeme ako **int** (celé číslo), **double** (reálne číslo), **char** (znak), **bool** (pravdivostná hodnota) a **adresa** (univerzálna referencia). Pre vytvorenie referenčnej premennej na určitý typ využívame zápis  $\uparrow \text{Typ}$ . Pre získanie adresy premennej používame funkciu **dajAdresu** (napr. **dajAdresu(id)**), pre získanie hodnoty z referenčnej premennej používame operátor  $\downarrow$  za premennou. Nepoužitú adresu vyjadrujeme literálom **NULL**. Pre priradenie hodnoty do premennej budeme používať znak  $\Leftarrow$ , pre sprístupnenie hodnoty zo štruktúrovaného typu (napr. objektu) operátor  $\rightarrow$ , pričom nerozlišujeme, či sa jedná o sprístupnenie priamo zo štruktúrovaného typu alebo o sprístupnenie pomocou referenčnej premennej naň (rozlíšenie tohto kontextu nechávame na čitateľa a spôsob implementácie v cieľovom jazyku). Použijeme teda jednotne zápis **objekt** $\rightarrow$ *vlastnosť* alebo **referenciaNaObjekt** $\rightarrow$ *vlastnosť*. Obdobne, pre vyvolanie operácie objektu či už priamo alebo pomocou referenčnej premennej využijeme operátor  $\rightarrow$ . Jednotný zápis potom vyzerá nasledovne **objekt** $\rightarrow$ **operácia()** resp. **referenciaNaObjekt** $\rightarrow$ **operácia()**.

Algebraické výrazy tvoríme pomocou operátorov  $+$ ,  $-$ ,  $*$  a  $/$  (v prípade neceločíselného delenia), **div** (v prípade celočíselného delenia) a **mod** (zvyšok po celočíselnom delení). Logické výrazy spájame logickými operátormi  $\wedge$  (a) a  $\vee$  (alebo). Negáciu logického výrazu robíme operátorom  $\neg$  pred logickým výrazom. Pre relačné výrazy využívame relačné operátory  $=$ ,  $\neq$ ,  $>$ ,  $\geq$ ,  $<$ ,  $\leq$ . Príklad použitia výrazov:

```
1. operácia jeRokPriestupný(rok: int): bool {  
2.   Vrát' rok mod 4 = 0  $\vee$  (rok mod 100  $\neq$  0  $\wedge$  rok mod 400 = 0)  
3. }
```

Triedy budeme označovať kľúčovým slovom **Trieda** nasledovaným jej identifikátorom (napr. **Trieda Auto**), rozhrania budeme označovať kľúčovým slovom **Rozhranie** nasledovaným jeho identifikátorom (napr. **Rozhranie Pohybivý**). Dedičnosť vyjadrujeme kľúčovým slovom **rozširuje** nasledovaným identifikátorom predka. Pre vyjadrenie realizácie rozhrania využívame kľúčové slovo **realizuje** nasledované identifikátorom rozhrania. Hlavička šablónovej triedy (rozhrania) pridáva za identifikátor triedy (rozhrania) zoznam šablónových parametrov oddelených čiarkami v  $\langle \rangle$  (napr. **Zoznam<T>**). Konštruktor budeme označovať kľúčovým slovom **konštruktor** a budeme ho volať kľúčovým slovom **vytvor** nasledovaným názvom triedy (napr. **vytvor Strom()**). Na označenie deštruktora využívame kľúčové slovo **deštruktor**. Na jeho zavolanie používame kľúčové slovo **zruš** nasledované identifikátorom inštancie (napr. **zruš stromček**).

Pre jednoduchosť nerozlišujeme v triedach úroveň prístupu k jej vlastnostiam a operáciám. Samozrejme, korektná realizácia pseudokódu v cieľovom programovacom jazyku predpokladá ich použitie a doplnenie prístupových metód k vhodným vlastnostiam. Vlastnosti objektov

označujeme kľúčovým slovom **vlastnosť** nasledovaným identifikátorom a dvojbodkou oddeleným typom, napr. **vlastnosť** *id*: **int**.

Operácie budeme označovať kľúčovým slovom **operácia** nasledovaným identifikátorom a povinnými parametrickými zátvorkami. Parametre operácií tvorí dvojbodkou oddelená dvojica identifikátora a typu parametra. Parametre navzájom oddeľujeme čiarkou. Ak má operácia návratovú hodnotu, jej typ je uvedený za parametrickými zátvorkami a je oddelený dvojbodkou. Návratová hodnota je z metódy odovzdaná príkazom **Vrát'**, ktorý ihneď skončí jej vykonávanie. Ak v operácii dôjde k chybe, označíme to volaním **CHYBA**, ktoré okamžite ukončí vykonávanie danej operácie (napr. vyvolaním výnimky). Príklad hlavičky operácie: **operácia** *vynásob*(*činiteľA*: **double**, *činiteľB*: **double**): **double**. Telo metódy je jednoznačne určené dvojicou zložených zátvoriek {}. Lokálne premenné sa definujú v bloku, ktorý určuje ich platnosť. Definícia vyzerá podobne ako definícia parametrov operácií alebo vlastností tried. Definícii predchádzajú kľúčové slová **definuj premennú** nasledované identifikátorom premennej, dvojbodkou oddeleným typom a prípadne inicializáciou. Príklad definície inicializácie lokálnej premennej: **definuj premennú** *sucet*: **int**  $\Leftarrow$  0. Operácie definujeme ako:

- **členské** operácie tried (jedná sa o metódu objektu). Hlavička tejto operácie je doplnená o identifikátor triedy.
- ako **nečlenské** operácie tried (teda operácia neparí žiadnej triede)
- ako operácie v iných (členských aj nečlenských) operáciách (takáto operácia je dostupná iba v tejto operácii a pozná kontext tejto operácie)
- ako **anonymné** operácie (posielané ako parameter typu operácia do inej operácie, poznajú kontext metódy, v ktorej sú definované). Príklad je uvedený nižšie:

```
1. operácia vykonaj(  
2.   a: int,  
3.   b: int,  
4.   akcia:  $\lambda(\text{int}, \text{int}) \rightarrow \text{int}$   
5. ): int {  
6.   Vrát' akcia(a, b)  
7. }  
8.  
9. operácia otestuj() {  
10.  definuj premennú a: int  $\Leftarrow$  5  
11.  definuj premennú b: int  $\Leftarrow$  10  
12.  definuj premennú c: int  $\Leftarrow$   
13.    vykonaj(a, b,  $\lambda(x: \text{int}, y: \text{int})$  Vrát' x + y )  
14.  definuj premennú d: int  $\Leftarrow$   
15.    vykonaj(c, 10,  $\lambda(x: \text{int}, y: \text{int})$  Vrát' x * y )  
16. }
```

V pseudokóde rozlišujeme úplné, neúplné a viacnásobné vetvenie. Neúplné vetvenie budeme označovať kľúčovým slovom **Ak** nasledovaným logickým výrazom v zátvorke, za zátvorkou nasleduje kľúčové slovo **potom** a blok určujúci sekvenciu príkazov, ktoré sa vykonajú, ak logický výraz nadobúda pravdivú hodnotu (**pravda**). Úplné vetvenie ďalej využíva vetvu **inak**

nasledovanú blokom, v ktorom sa nachádza sekvencia príkazov, ktoré sa vykonajú, ak logický výraz nadobúda nepravdivú hodnotu (**nepravda**). Viacnásobné vetvenie označujeme kľúčovým slovom **Ked'** nasledovaným ordinálnym výrazom v zátvorke a kľúčovým slovom **nadobúda** a telom. V tele viacnásobného vetvenia sa nachádzajú bloky uvedené kľúčovým slovom **hodnotu** nasledovaným čiarkami oddelenými možnými hodnotami ordinálneho výrazu a kľúčovým slovom **tak**. Telo tohto bloku sa vykoná, ak ordinálny výraz nadobúda ktorúkoľvek z uvedených možných hodnôt. Vo viacnásobnom vetvení môže byť uvedený blok kľúčovými slovami **žiadnu z uvedených hodnôt**, ktorý sa vykoná, ak výraz nenadobúda žiadnu z vymenovaných hodnôt v blokoch viacnásobného vetvenia. Príklad vetvení uvádzame nižšie.

```
1. operácia početDníVMesiaci(mesiac: int, rok: int): int {  
2.   Ked' (mesiac) nadobúda {  
3.     hodnotu 1, 3, 5, 7, 8, 10, 12 tak {  
4.       Vrát' 31;  
5.     }  
6.     hodnotu 4, 6, 9, 11 tak {  
7.       Vrát' 30;  
8.     }  
9.     hodnotu 2 tak {  
10.      Ak (JeRokPriestupný(rok)) potom {  
11.        Vrát' 29  
12.      }  
13.      inak {  
14.        Vrát' 28  
15.      }  
16.    }  
17.    žiadnu z uvedených hodnôt {  
18.      CHYBA  
19.    }  
20.  }  
21. }
```

V pseudokódoch budeme používať cyklus s podmienkou na začiatku, cyklus s podmienkou na konci a cyklus s pevným počtom opakovaní. Cyklus s podmienkou na začiatku označujeme kľúčovým slovom **Pokiaľ'** nasledovaným logickým výrazom v zátvorke, kľúčovým slovom **opakuj** a telom. Ak podmienka platí, vykoná sa telo cyklu a pokračuje sa opätovným vyhodnotením podmienky a prípadným opakovaným vykonaním tela. Cyklus s podmienkou na konci uvádzame kľúčovým slovom **Opakuj** nasledovaným telom cyklu, kľúčovým slovom **pokiaľ'** a logickým výrazom v zátvorke. Telo cyklu sa vždy vykoná aspoň raz, následne sa vyhodnotí podmienka a ak je splnená, telo sa opätovne vykoná. Cyklus s pevným počtom opakovaní označujeme kľúčovými slovami **Opakuj pre premennú** nasledovanými uvedením riadiacej premennej cyklu (spojenou s jej prípadnou definíciou), kľúčovým slovom **od** nasledovaným počiatočnou hodnotou, kľúčovým slovom **do** nasledovaným koncovou hodnotou a telom cyklu. Riadiaca premenná bude postupne nadobúdať všetky hodnoty z uvedeného rozsahu (vrátane konca rozsahu), pričom telo cyklu sa vykoná postupne pre každú jej hodnotu. Príklady cyklov uvádzame nižšie:

```
1. operácia faktoriálA(n: int): int {
2.   Ak (n < 0) potom { CHYBA }
3.   definuj premennú výsledok: int  $\leftarrow$  1
4.   Pokiaľ (n > 1) opakuj {
5.     výsledok  $\leftarrow$  výsledok * n
6.     n  $\leftarrow$  n - 1
7.   }
8.   Vráť výsledok
9. }
10.
11. operácia faktoriálB(n: int): int {
12.   Ak (n < 0) potom { CHYBA }
13.   definuj premennú výsledok: int  $\leftarrow$  1
14.   Opakuj {
15.     výsledok  $\leftarrow$  výsledok * n
16.     n  $\leftarrow$  n - 1
17.   } pokiaľ (n > 1)
18.   Vráť výsledok
19. }
20.
21. operácia faktoriálC(n: int): int {
22.   Ak (n < 0) potom { CHYBA }
23.   definuj premennú výsledok: int  $\leftarrow$  1
24.   Opakuj pre premennú i: int od 2 do n {
25.     výsledok  $\leftarrow$  výsledok * i
26.   }
27.   Vráť výsledok
28. }
```

V pseudokódoch sa budeme vyhýbať pretypovaniu. Ak predok definuje atribút typu (referencia na) "rozhranie" alebo "trieda" a potomok má znalosť o skutočnom type alebo o konkrétnejšom potomkovi triedy/rozhrania, budeme v pseudokódoch potomka pracovať s týmto typom.